

GUÍA DE TRANSAQ - SQL (T-SQL)

Índice

1. Introducción: SQL vs T-SQL y entorno de trabajo
 2. Áreas donde añadir explicaciones profundas (diagnóstico previo)
 3. Creación de la base de datos — explicación y riesgos
 4. Diseño de tablas: claves, tipos y normalización (detallado)
 5. Creación de tablas y constraints
 6. ALTER TABLE: estrategia segura para cambios cuando hay datos
 7. INSERT: buenas prácticas, validación y manejo por lotes
 8. UPDATE y DELETE: patrones seguros, OUTPUT y auditoría
 9. Relaciones e integridad referencial — elección de ON DELETE / ON UPDATE
 10. Transacciones y TRY...CATCH — ACID y manejo robusto de errores
 11. Consultas JOIN y diagnóstico de inconsistencias (órfanos, duplicados)
 12. Metadatos y exploración del esquema (herramientas T-SQL)
 13. Rendimiento básico: índices, plan de ejecución y trade-offs
 14. Backup y restauración: tipos y cuándo usarlos (práctica educativa)
 15. Concurrencia, niveles de aislamiento y deadlocks (introducción)
 16. Seguridad: principio de mínimos privilegios y prácticas
 17. Errores frecuentes — lista diagnóstica y cómo enseñarlos
 18. Actividades prácticas y criterios de evaluación (guía para el docente)
 19. Script completo (plantilla ejecutable ordenada)
 20. Cierre y recomendaciones de entrega
-

1. Introducción: SQL vs T-SQL y entorno de trabajo

Concepto:

- **SQL** es el estándar (ISO) para manipulación y definición de datos: **SELECT**, **INSERT**, **UPDATE**, **DELETE**, **CREATE**, **ALTER**, **DROP**.
- **T-SQL** (Transact-SQL) es la extensión de Microsoft SQL Server que añade: control procedural (**IF**, **WHILE**), manejo de errores (**TRY...CATCH**), variables, **sp_***, y utilidades propias (**GO**, **sys.***).
- Sintaxis portable vs extensiones específicas: por ejemplo **LIMIT** (MySQL) **no** funciona en SQL Server; en cambio, **TOP** o **OFFSET/FETCH** son las alternativas.
- **GO** no es una instrucción del motor: es un separador de lotes que interpreta el cliente (SSMS). aprender esto evita malentendidos.

Actividad corta (diagnóstico): ejecutar **SELECT @@VERSION**; en SSMS y revisar la versión del servidor para saber qué funciones T-SQL están disponibles.

2. Áreas donde conviene añadir explicaciones profundas (diagnóstico previo)

He identificado las siguientes áreas donde la explicación debe ser explícita y profunda para evitar errores conceptuales y operativos:

- **Elección de claves:** natural vs surrogate — impacto en integridad y rendimiento.
- **Tipos de datos:** CHAR vs VARCHAR vs NVARCHAR, fechas (**DATE** vs **DATETIME**), campos numéricos.
- **Alteraciones de esquema con datos existentes:** patrón seguro para evitar pérdida de información.
- **Políticas de integridad referencial (**ON DELETE/ON UPDATE**):** implicaciones de negocio.
- **Transacciones y manejo de errores:** **TRY...CATCH**, **XACT_STATE()**.
- **Interpretación de errores** de SQL Server y pasos diagnósticos reproducibles en clase.
- **Índices y trade-offs** entre lectura y escritura.
- **Backups y restauración** en entorno educativo para permitir reset de prácticas.

Cada sección más abajo desarrolla con ejemplos y preguntas guía para los estudiantes.

3. Creación de la base de datos — explicación y riesgos

Qué ocurre con **CREATE DATABASE:** crea el catálogo lógico y los archivos físicos (**.mdf**, **.ldf**) en las rutas configuradas del servidor. Requiere permisos suficientes (**CREATE DATABASE**) y espacio en disco.

Puntos de enseñanza:

- En un laboratorio usar configuración por defecto; en producción siempre definir tamaño inicial, autogrowth y rutas.
- **GO** separa lotes; si se crea un objeto y se desea usar inmediatamente, **GO** asegura el orden de ejecución.

Código de ejemplo:

```
CREATE DATABASE UniversidadUNP;  
GO  
  
USE UniversidadUNP;  
GO
```

Diagnóstico de fallos comunes:

- Error por permisos: revisar rol del usuario.
 - Error por falta de espacio: comprobar volumen del servidor.
-

4. Diseño de tablas: claves, tipos y normalización (detallado)

4.1 Claves: natural vs surrogate

- **Clave natural:** valor con significado del dominio (ej. **CodigoCarrera = 'ISI001'**). *Pros:* legible. *Contras:* puede cambiar, largo, puede contener significado mutable.

- **Clave surrogate:** `INT IDENTITY`. *Pros:* pequeña, estable, eficiente en índices. *Contras:* sin significado humano.
- **Recomendación pedagógica:** usar `Id*` (surrogate) para relaciones y mantener `Codigo*` natural con `UNIQUE` para reportes.

4.2 Tipos de datos

- `CHAR(n)` para longitudes fijas (códigos).
- `VARCHAR(n)` para texto variable; `NVARCHAR(n)` si necesita Unicode (acentos). *En Nicaragua, preferir NVARCHAR para nombres y descripciones.*
- `DATE` si solo requiere fecha; `DATETIME2` o `DATETIME` si necesita precisión horaria.
- Enteros: `TINYINT`, `SMALLINT`, `INT` según rango. Evitar `VARCHAR(255)` por defecto.

4.3 Normalización (1NF, 2NF, 3NF)

- 1NF: cada columna atómica.
 - 2NF: eliminar dependencias parciales cuando exista clave compuesta.
 - 3NF: eliminar dependencias transitivas.
- Ejercicio:** identificar atributos que podrían extraerse a tablas separadas (ej. sedes, direcciones).

5. Creación de tablas y constraints

Script comentado (crear tablas padres antes de hijos):

```
-- Tabla Facultad (padre)
CREATE TABLE Facultad (
    IdFacultad INT IDENTITY(1,1) PRIMARY KEY,           -- surrogate key
    CodigoFacultad CHAR(6) NOT NULL UNIQUE,             -- código legible, p.ej.
    'FAC001'
    Nombre NVARCHAR(150) NOT NULL
);
GO

-- Tabla Carrera (hija de Facultad)
CREATE TABLE Carrera (
    IdCarrera INT IDENTITY(1,1) PRIMARY KEY,
    CodigoCarrera CHAR(6) NOT NULL UNIQUE,
    Nombre NVARCHAR(150) NOT NULL,
    DuracionAnios TINYINT NOT NULL CHECK (DuracionAnios BETWEEN 1 AND 10),
    IdFacultad INT NOT NULL,
    CONSTRAINT FK_Carrera_Facultad FOREIGN KEY (IdFacultad) REFERENCES
Facultad(IdFacultad)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION
);
GO

-- Tabla Estudiante (hija de Carrera)
CREATE TABLE Estudiante (
    Carnet CHAR(8) PRIMARY KEY,                         -- formato YYYYNNNN (ej:
```

```

20230001)
    Nombre NVARCHAR(50) NOT NULL,
    Apellido NVARCHAR(50) NOT NULL,
    FechaIngreso DATE NOT NULL,
    Procedencia NVARCHAR(100), -- sede: Rivas, Managua,
Boaco, Estelí
    Telefono VARCHAR(15) NULL,
    IdCarrera INT NOT NULL,
    CONSTRAINT FK_Estudiante_Carrera FOREIGN KEY (IdCarrera) REFERENCES
Carrera(IdCarrera)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION
);
GO

-- Tabla Asignatura (para relacionar con Estudiante)
CREATE TABLE Asignatura (
    IdAsignatura INT IDENTITY(100,1) PRIMARY KEY,
    CodigoAsignatura CHAR(6) NOT NULL UNIQUE,
    NombreAsignatura NVARCHAR(150) NOT NULL,
    Creditos TINYINT NOT NULL CHECK (Creditos > 0)
);
GO

-- Tabla Matricula (N:M Estudiante-Asig)
CREATE TABLE Matricula (
    IdMatricula INT IDENTITY(1,1) PRIMARY KEY,
    Carnet CHAR(8) NOT NULL,
    IdAsignatura INT NOT NULL,
    FechaMatricula DATE NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT FK_Matricula_Estudiante FOREIGN KEY (Carnet) REFERENCES
Estudiante(Carnet) ON DELETE CASCADE,
    CONSTRAINT FK_Matricula_Asignatura FOREIGN KEY (IdAsignatura) REFERENCES
Asignatura(IdAsignatura) ON DELETE NO ACTION
);
GO

```

Notas:

- Explique por qué se usan **CHECK** (valida rango) y **UNIQUE** (evita duplicados lógicos).
- Indique que **ON DELETE CASCADE** en **Matricula** es cómodo para datos de laboratorio, pero debe usarse con cautela en producción.

6. ALTER TABLE: estrategia segura para cambios cuando existen datos

Problema frecuente: **ALTER COLUMN** falla por conversiones inválidas.

Patrón seguro (paso a paso):

1. Añadir columna temporal con nuevo tipo.
2. Población y transformación controlada usando funciones (p.ej. **REPLACE**, **RIGHT**, **TRY_CONVERT**).
3. Inspección de filas problemáticas (**SELECT** para detectar **NULL** o longitudes inválidas).

4. Si todo OK, eliminar columna original y renombrar la temporal.
5. Actualizar documentación y objetos dependientes.

Ejemplo completo:

```
-- 1) Añadir columna temporal
ALTER TABLE Estudiante ADD Telefono_tmp CHAR(8) NULL;
GO

-- 2) Copiar datos normalizados
UPDATE Estudiante
SET Telefono_tmp = RIGHT(REPLACE(REPLACE(Telefono, '-', ''), ' ', ''), 8)
WHERE Telefono IS NOT NULL;
GO

-- 3) Revisar problemas
SELECT Carnet, Telefono, Telefono_tmp
FROM Estudiante
WHERE Telefono IS NOT NULL AND (Telefono_tmp IS NULL OR LEN(Telefono_tmp) <> 8);
GO

-- 4) Tras validación humana:
ALTER TABLE Estudiante DROP COLUMN Telefono;
GO
EXEC sp_rename 'Estudiante.Telefono_tmp', 'Telefono', 'COLUMN';
GO
```

Consejos: documentar cada paso en el script (-- comentarios), y no ejecutar en producción sin backup.

7. INSERT: buenas prácticas, validación y manejo por lotes

Buenas prácticas para INSERT:

- Especificar la lista de columnas en la instrucción.
- Validar FK existentes antes de inserciones masivas.
- Para cargas grandes usar **BULK INSERT** o **OPENROWSET** (configuración y permisos necesarios).
- Usar transacciones para garantizar que lotes críticos se hagan de forma atómica.

Ejemplo de inserciones contextualizadas:

```
INSERT INTO Facultad (CodigoFacultad, Nombre) VALUES
('FAC001', 'Ciencias y Tecnología'),
('FAC002', 'Servicios'),
('FAC003', 'Ciencias Económicas');
GO

INSERT INTO Carrera (CodigoCarrera, Nombre, DuracionAnios, IdFacultad) VALUES
('ISI001', 'Ingeniería en Sistemas de Información', 5, 1),
('ENF001', 'Licenciatura en Enfermería', 5, 2),
```

```

('CON001','Contaduría Pública y Finanzas',5,3);
GO

INSERT INTO Estudiante (Carnet, Nombre, Apellido, FechaIngreso, Procedencia,
Telefono, IdCarrera) VALUES
('20230001','María','López','2023-02-10','Rivas','88881234',1),
('20230002','Juan','Martínez','2023-02-11','Managua','88885678',2),
('20230003','Ana','García','2023-02-12','Boaco','88889012',3);
GO

INSERT INTO Asignatura (CodigoAsignatura, NombreAsignatura, Creditos) VALUES
('PROG01','Programación I',4),
('MATE01','Matemáticas I',3),
('CONT01','Contabilidad Básica',3);
GO

INSERT INTO Matricula (Carnet, IdAsignatura, FechaMatricula) VALUES
('20230001', 100, '2023-02-15'),
('20230001', 101, '2023-02-15'),
('20230003', 102, '2023-02-16');
GO

```

Errores frecuentes y diagnóstico:

- **Violation of PRIMARY KEY**: hacer **SELECT** previo para detectar existencias.
- **FOREIGN KEY constraint violation**: comprobar la existencia en la tabla padre.
- Formato de fecha ambiguo: siempre usar **YYYY-MM-DD**.

8. UPDATE y DELETE: patrones seguros, OUTPUT y auditoría

Patrones seguros:

- Ejecutar **SELECT** con el mismo **WHERE** antes de **UPDATE** o **DELETE**.
- En operaciones críticas, envolver en **BEGIN TRAN / COMMIT / ROLLBACK**.
- Usar **OUTPUT** para capturar las filas afectadas para auditoría.

Ejemplos:

```

-- Verificar antes de actualizar
SELECT * FROM Estudiante WHERE Carnet = '20230001';

BEGIN TRAN;
UPDATE Estudiante
SET Telefono = '77771234'
WHERE Carnet = '20230001';
-- Revisar resultado
SELECT * FROM Estudiante WHERE Carnet = '20230001';
COMMIT;
GO

```

```
-- DELETE con OUTPUT
DELETE FROM Matricula
OUTPUT DELETED.IdMatricula, DELETED.Carnet, DELETED.IdAsignatura
WHERE IdMatricula = 3;
GO
```

Diagnóstico docente: diseñar un ejercicio donde un estudiante ejecute por error **DELETE** sin **WHERE** y luego demostrar recuperación a partir de backup o tabla de auditoría.

9. Relaciones e integridad referencial — elección de ON DELETE / ON UPDATE

Opciones y consideraciones (resumen):

- **ON DELETE CASCADE:** elimina dependientes automáticamente. Útil en datos temporales; peligroso si la regla de negocio no lo requiere.
- **ON DELETE SET NULL:** adecuado si la relación es opcional.
- **ON DELETE NO ACTION / RESTRICT:** impide eliminación hasta que dependientes se limpien explícitamente.

Ejemplo de cambio de política:

```
ALTER TABLE Carrera
DROP CONSTRAINT FK_Carrera_Facultad;
GO

ALTER TABLE Carrera
ADD CONSTRAINT FK_Carrera_Facultad FOREIGN KEY (IdFacultad)
REFERENCES Facultad(IdFacultad)
ON DELETE NO ACTION;
GO
```

Recomendación práctica: mapear política de integridad a reglas del negocio y documentar.

10. Transacciones y TRY...CATCH — ACID y manejo robusto de errores

Conceptos clave:

- **Atomicidad:** todas las operaciones de la transacción se aplican o ninguna.
- **Consistencia:** la BD pasa de un estado válido a otro.
- **Isolation:** las transacciones concurrentes no interfieren (según nivel).
- **Durability:** cambios confirmados persisten.

Ejemplo robusto con diagnóstico:

```

BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO Estudiante (Carnet, Nombre, Apellido, FechaIngreso, Procedencia,
Telefono, IdCarrera)
    VALUES ('20230004','Carlos','Ramírez','2023-02-13','Estelí','88882345',1);

    -- Intento de insertar matrícula (posible FK violation si Asignatura no
existe)
    INSERT INTO Matricula (Carnet, IdAsignatura) VALUES ('20230004', 100);

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0
        ROLLBACK TRANSACTION;

    DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrNo INT = ERROR_NUMBER();
    DECLARE @ErrLine INT = ERROR_LINE();

    PRINT 'Error N° ' + CAST(@ErrNo AS VARCHAR(10)) + ' en línea ' + CAST(@ErrLine
AS VARCHAR(10)) + ': ' + @ErrMsg;
    -- Opcional: insertar en tabla de logs
END CATCH;
GO

```

Diagnóstico común: olvidar **COMMIT** deja la transacción abierta (bloqueos). Use **XACT_STATE()** para entender el estado de la transacción y si se puede reintentar.

11. Consultas JOIN y diagnóstico de inconsistencias (órfanos, duplicados)

Mostrar estudiante + carrera + facultad:

```

SELECT E.Carnet, E.Nombre, E.Apellido, C.Nombre AS Carrera, F.Nombre AS Facultad
FROM Estudiante E
JOIN Carrera C ON E.IdCarrera = C.IdCarrera
JOIN Facultad F ON C.IdFacultad = F.IdFacultad;
GO

```

Detectar órfanos en Matricula (LEFT JOIN pattern):

```

SELECT M.*
FROM Matricula M
LEFT JOIN Estudiante E ON M.Carnet = E.Carnet
WHERE E.Carnet IS NULL;
GO

```


Detectar duplicados lógicos:

```
SELECT Carnet, IdAsignatura, FechaMatricula, COUNT(*) AS Cantidad
FROM Matricula
GROUP BY Carnet, IdAsignatura, FechaMatricula
HAVING COUNT(*) > 1;
GO
```

Enseñanza: explicar por qué **LEFT JOIN + IS NULL** encuentra registros dependientes inválidos; usar **GROUP BY + HAVING** para reglas de unicidad lógicas que no están impuestas por constraint.

12. Metadatos y exploración del esquema (herramientas T-SQL)

Columnas y tipos de una tabla:

```
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE, CHARACTER_MAXIMUM_LENGTH
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_NAME = 'Estudiante';
GO
```

Listar llaves foráneas que referencian una tabla:

```
SELECT fk.name AS FK_Name, OBJECT_NAME(fk.parent_object_id) AS TableName
FROM sys.foreign_keys fk
WHERE fk.referenced_object_id = OBJECT_ID('Carrera');
GO
```

Diagnóstico: siempre inspeccionar dependencias antes de **DROP** o **ALTER** para evitar errores por objetos dependientes (views, SPs, funciones).

13. Rendimiento básico: índices, plan de ejecución y trade-offs

Principio técnico: un índice B-tree acelera búsqueda y join, pero penaliza operaciones que modifican datos (INSERT/UPDATE/DELETE) y consume espacio.

Ejemplo de índice no clusterizado:

```
CREATE NONCLUSTERED INDEX IX_Estudiante_IdCarrera ON Estudiante(IdCarrera);
GO
```

Actividad diagnosticable: usar **SET STATISTICS IO ON; SET STATISTICS TIME ON;** antes y después de crear índice para comparar lecturas físicas y tiempos.

Recomendación: revisar plan de ejecución (**Actual Execution Plan** en SSMS) para detectar scans costosos y proponer índices adecuados.

14. Backup y restauración — tipos y cuándo usarlos

Tipos de backup:

- **Full:** copia completa de la BD.
- **Differential:** cambios desde el último full.
- **Transaction log:** secuencia de transacciones para recuperación punto-en-tiempo.

Ejemplo de backup (ruta del servidor):

```
BACKUP DATABASE UniversidadUNP TO DISK = 'C:\Backups\UniversidadUNP.bak';  
GO
```

Recomendación operativa: en entorno de prácticas mantener una "imagen base" para restaurar antes de cada sesión de laboratorio, permitiendo repetir ejercicios sin contaminación por cambios previos.

15. Concurrencia, niveles de aislamiento y deadlocks (introducción)

Niveles de aislamiento (resumen):

- **READ UNCOMMITTED:** lecturas sucias permitidas.
- **READ COMMITTED** (default): evita lecturas sucias.
- **REPEATABLE READ:** evita que una fila leída cambie.
- **SERIALIZABLE:** máxima restricción (más bloqueos).
- **SNAPSHOT:** lectura consistente mediante versionado (requiere habilitar).

Deadlocks: dos transacciones se bloquean mutuamente esperando recursos. Enseñar a reproducir un deadlock controlado y a usar herramientas (**system_health**, **sys.dm_os_waiting_tasks**) para diagnóstico.

16. Seguridad: principio de mínimos privilegios y prácticas

Recomendaciones:

- No ejecutar scripts con **sa** en producción.
 - Crear roles (**db_docente**, **db_alumno**) y otorgar permisos mínimos (**GRANT SELECT, INSERT, UPDATE**).
 - Evitar SQL dinámico concatenado; usar parámetros para prevenir SQL Injection.
-

17. Errores frecuentes — lista diagnóstica y cómo enseñarlos

1. **Msg 2627 — PRIMARY KEY violation:** detectar con **SELECT** previo; decidir **UPDATE** o ignorar.
2. **Msg 547 — FOREIGN KEY constraint violation:** revisar existencia del padre y el orden de inserción.
3. **Conversion failed** al **ALTER COLUMN:** usar estrategia de columna temporal y validación.

4. **Bloqueos por transacciones abiertas:** revisar con `sp_who2` y cerrar transacciones.
5. **Borrado masivo por omisión de WHERE:** siempre revisar con `SELECT` previo y tener backup.

Actividad didáctica: dar un script con errores intencionales para que los estudiantes diagnostiquen y corrijan, documentando cada paso.

18. Actividades prácticas y criterios de evaluación (guía docente)

Actividad A — Implementación básica (individual)

- Crear BD y tablas (`Facultad`, `Carrera`, `Estudiante`).
- Insertar 5 facultades, 8 carreras, 20 estudiantes (sedes distribuidas).
- Entregar consulta que liste estudiantes con su carrera y facultad.

Actividad B — ALTER seguro (pareja)

- Normalizar teléfonos con formatos mixtos a `CHAR(8)` usando patrón seguro. Documentar pasos y filas problemáticas.

Actividad C — Transacción (grupo)

- Script que inserte estudiante + 3 matrículas en transacción. Inducir fallo (p.ej. FK inválida) y demostrar `ROLLBACK`.

Criterios de evaluación (sin puntaje):

- Código correcto y comentado.
 - Uso de transacciones en operaciones críticas.
 - Capacidad de diagnosticar y resolver inconsistencias.
 - Claridad en la documentación de decisiones de diseño.
-

19. Script completo — ejecutar paso a paso

Ejecutar bloque por bloque en SSMS y revisar resultados antes de continuar.

```
/* --- CREAR BD Y SELECCIONAR CONTEXTO --- */
CREATE DATABASE UniversidadUNP;
GO
USE UniversidadUNP;
GO

/* --- CREAR TABLAS --- */
CREATE TABLE Facultad (
    IdFacultad INT IDENTITY(1,1) PRIMARY KEY,
    CodigoFacultad CHAR(6) NOT NULL UNIQUE,
    Nombre NVARCHAR(150) NOT NULL
);
GO
```

```

CREATE TABLE Carrera (
    IdCarrera INT IDENTITY(1,1) PRIMARY KEY,
    CodigoCarrera CHAR(6) NOT NULL UNIQUE,
    Nombre NVARCHAR(150) NOT NULL,
    DuracionAnios TINYINT NOT NULL CHECK (DuracionAnios BETWEEN 1 AND 10),
    IdFacultad INT NOT NULL,
    CONSTRAINT FK_Carrera_Facultad FOREIGN KEY (IdFacultad) REFERENCES
Facultad(IdFacultad)
);
GO

CREATE TABLE Estudiante (
    Carnet CHAR(8) PRIMARY KEY,
    Nombre NVARCHAR(50) NOT NULL,
    Apellido NVARCHAR(50) NOT NULL,
    FechaIngreso DATE NOT NULL,
    Procedencia NVARCHAR(100),
    Telefono VARCHAR(15) NULL,
    IdCarrera INT NOT NULL,
    CONSTRAINT FK_Estudiante_Carrera FOREIGN KEY (IdCarrera) REFERENCES
Carrera(IdCarrera)
);
GO

CREATE TABLE Asignatura (
    IdAsignatura INT IDENTITY(100,1) PRIMARY KEY,
    CodigoAsignatura CHAR(6) NOT NULL UNIQUE,
    NombreAsignatura NVARCHAR(150) NOT NULL,
    Creditos TINYINT NOT NULL CHECK (Creditos > 0)
);
GO

CREATE TABLE Matricula (
    IdMatricula INT IDENTITY(1,1) PRIMARY KEY,
    Carnet CHAR(8) NOT NULL,
    IdAsignatura INT NOT NULL,
    FechaMatricula DATE NOT NULL DEFAULT (GETDATE()),
    CONSTRAINT FK_Matricula_Estudiante FOREIGN KEY (Carnet) REFERENCES
Estudiante(Carnet) ON DELETE CASCADE,
    CONSTRAINT FK_Matricula_Asignatura FOREIGN KEY (IdAsignatura) REFERENCES
Asignatura(IdAsignatura) ON DELETE NO ACTION
);
GO

/* --- INSERTS DE EJEMPLO (contexto Nicaragua) --- */
INSERT INTO Facultad (CodigoFacultad, Nombre) VALUES
('FAC001', 'Ciencias y Tecnología'),
('FAC002', 'Servicios'),
('FAC003', 'Ciencias Económicas');
GO

INSERT INTO Carrera (CodigoCarrera, Nombre, DuracionAnios, IdFacultad) VALUES
('ISI001', 'Ingeniería en Sistemas de Información', 5, 1),
('ENF001', 'Licenciatura en Enfermería', 5, 2),

```

```

('CON001','Contaduría Pública y Finanzas',5,3);
GO

INSERT INTO Estudiante (Carnet, Nombre, Apellido, FechaIngreso, Procedencia, Telefono, IdCarrera) VALUES
('20230001','María','López','2023-02-10','Rivas','88881234',1),
('20230002','Juan','Martínez','2023-02-11','Managua','88885678',2),
('20230003','Ana','García','2023-02-12','Boaco','88889012',3);
GO

INSERT INTO Asignatura (CodigoAsignatura, NombreAsignatura, Creditos) VALUES
('PROG01','Programación I',4),
('MATE01','Matemáticas I',3),
('CONT01','Contabilidad Básica',3);
GO

INSERT INTO Matricula (Carnet, IdAsignatura, FechaMatricula) VALUES
('20230001', 100, '2023-02-15'),
('20230001', 101, '2023-02-15'),
('20230003', 102, '2023-02-16');
GO

/* --- CONSULTAS DE DIAGNÓSTICO --- */
SELECT E.Carnet, E.Nombre, E.Apellido, C.Nombre AS Carrera, F.Nombre AS Facultad
FROM Estudiante E
JOIN Carrera C ON E.IdCarrera = C.IdCarrera
JOIN Facultad F ON C.IdFacultad = F.IdFacultad;
GO

-- Detectar órfanos en Matricula
SELECT M.*
FROM Matricula M
LEFT JOIN Estudiante E ON M.Carnet = E.Carnet
WHERE E.Carnet IS NULL;
GO

-- Detectar duplicados lógicos en Matricula
SELECT Carnet, IdAsignatura, FechaMatricula, COUNT(*) AS Cantidad
FROM Matricula
GROUP BY Carnet, IdAsignatura, FechaMatricula
HAVING COUNT(*) > 1;
GO

/* --- EJEMPLO ALTER SEGURO (convertir Telefono) --- */
ALTER TABLE Estudiante ADD Telefono_tmp CHAR(8) NULL;
GO

UPDATE Estudiante
SET Telefono_tmp = RIGHT(REPLACE(REPLACE(Telefono, '-', ''), ' ', ''), 8)
WHERE Telefono IS NOT NULL;
GO

SELECT Carnet, Telefono, Telefono_tmp
FROM Estudiante

```

```

WHERE Telefono IS NOT NULL AND (Telefono_tmp IS NULL OR LEN(Telefono_tmp) <> 8);
GO

-- Si ya validado:
ALTER TABLE Estudiante DROP COLUMN Telefono;
GO
EXEC sp_rename 'Estudiante.Telefono_tmp', 'Telefono', 'COLUMN';
GO

/* --- EJEMPLO TRANSACCIONAL CON TRY/CATCH --- */
BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO Estudiante (Carnet, Nombre, Apellido, FechaIngreso, Procedencia,
Telefono, IdCarrera)
    VALUES ('20230004', 'Carlos', 'Ramírez', '2023-02-13', 'Estelí', '88882345', 1);

    INSERT INTO Matricula (Carnet, IdAsignatura) VALUES ('20230004', 100);

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;

    DECLARE @ErrMsg NVARCHAR(4000) = ERROR_MESSAGE();
    DECLARE @ErrNo INT = ERROR_NUMBER();
    DECLARE @ErrLine INT = ERROR_LINE();

    PRINT 'Error N° ' + CAST(@ErrNo AS VARCHAR(10)) + ' en línea ' + CAST(@ErrLine
AS VARCHAR(10)) + ': ' + @ErrMsg;
END CATCH;
GO

/* --- LIMPIEZA: ejecutar SOLO si se desea eliminar todo --- */
/*
DROP TABLE Matricula;
DROP TABLE Asignatura;
DROP TABLE Estudiante;
DROP TABLE Carrera;
DROP TABLE Facultad;
DROP DATABASE UniversidadUNP;
GO
*/

```

20. Cierre y recomendaciones de entrega

- **Ejecución paso a paso:** en clase ejecutar por bloques y detenerse en los puntos de diagnóstico (errores intencionales para practicar).
- **Documentación:** exigir a los estudiantes comentarios en sus scripts explicando cada decisión.
- **Restauración:** mantener una copia base (.bak) para restaurar el entorno de prácticas rápidamente.
- **Siguientes materiales opcionales (puedo generarlos):** diagrama ER en PlantUML dentro del mismo bloque, archivo .sql descargable, hoja de ejercicios con solución y rúbrica.

